

RCM Alternatives Whitepaper

Machine Learning for Bitcoin Pricing Classification

Thematic Areas:

- ☐ Digital Asset Valuation
- ☐ Volatile Price Dynamics
- ☐ Machine Learning for Finance Time Series Forecasting
- ☐ Crypto Market Microstructure
- ☐ On-Chain Analytics
- ☐ Neural Networks and Deep Learning in Financial Markets
- ☐ Macroeconomic Factors
- ☐ Comparative Evaluation of ML Algorithms in Asset Prediction

Author: Sohan Bendre

RCM Alternatives Internship Program 2025

Sponsor: Ryan Dillman

Sponsor: Tara Begeman

Abstract

This paper follows the development process of various machine learning models, used to provide context on cryptocurrency pricing. Bitcoin is the first decentralized digital currency, built on blockchain technology to facilitate secure transactions without oversight from a central entity. However, Bitcoin prices are historically extremely volatile – driven by fluctuating supply and demand, along with relative novelty – and thus very difficult to predict. The innovative models in this paper *combine* data from various on-chain metrics (publicly-recorded blockchain data); indices, commodities, and Forex markets; and macroeconomic indicators to forecast the magnitude of Bitcoin pricing on a daily basis. The study compares the performance of four different machine learning models implemented through Python libraries scikit-learn and tensorflow: Logistic Regression, Support Vector Machines (SVM), Tree-Based Algorithms, and Long-Short Term Memory (LSTM) recurrent neural networks. Results are quantified based on accuracy score, with logistic regression providing the highest accuracy at 55.94% on out-of-sample daily data from March 2024 to May 2025, and statistical significance across the full period.

Introduction

Bitcoin, the first and most noteworthy cryptocurrency, is increasingly becoming a pivotal financial asset, since its creation in 2009. As a decentralized asset operated independently of traditional financial institutions, Bitcoin has attracted widespread attention from professionals across various fields. Its price behavior, however, is notoriously volatile and driven by a complex network of macroeconomic trends, market sentiment, on-chain activity, and more. This volatility, in conjunction with the absence of intrinsic value or standardized valuation models, has fostered skepticism around Bitcoin’s predictability. Yet really understanding Bitcoin’s price movements has high significance for portfolio optimization, trading and risk management, and gaining key insights into digital asset behavior. By leveraging some machine learning pipelines across a variety of financial and digital indicators, this project uncovers latent patterns in Bitcoin’s price movement.

Data Selection and Framework

Through data collection, the factors were categorized as either on-chain metrics, market data, or macroeconomic indicators.

List of Factors		
On-Chain Metrics	Market Data	Macroeconomic Indicators
Active Addresses	* S&P 500 Close	** Federal Funds Rate
Exchange Holdings	* Nasdaq Close	Consumer Price Index (CPI)
Exchange Netflow	* Dow Jones Close	** 10Y Treasury Yield
Fee-Reward Ratio	* Gold Close	Real GDP
Funding Rate	Crude Oil Close	M2 Money Supply
* Hashrate	Nat Gas Close	Unemployment Rate
* Mean Coin Age	* Silver Close	
* Miner Reserves	* Copper Close	
* MVRV Ratio	US Dollar Index	
* Open Interest	EUR-USD Exchange Rate	
Puell Multiple	USD-CNY Exchange Rate	
SOPR Ratio		

Table 1: Variables marked with * were selected for our models. Variables marked with ** additionally required linear interpolation.

On-Chain Data

The Importance of Understanding UTxOs

A clear grasp of Unspent Transaction Outputs (UTxOs) is essential for anyone analyzing the Bitcoin blockchain. UTxOs are fundamental components of the Bitcoin transaction model. Each UTxO represents a discrete amount of bitcoin that has not yet been spent, functioning similarly to holding cash in assorted denominations. When a transaction occurs, these outputs are either fully consumed or returned partially as change, effectively creating new UTxOs. This structure enables transparent, verifiable ownership and contributes to network security by preventing issues such as double-spending. UTxOs can only be spent in totality, reinforcing the atomic nature of bitcoin ownership and transfer. They are the essential building blocks for all on-chain metrics, which reflect the state of Bitcoin from many different angles.

On-Chain Data Visualization

This organization and selection of on-chain metrics provide an interpretable feature set for quantitatively modeling Bitcoin price movements. All on-chain data, INCLUDING Bitcoin close prices, were collected from CryptoQuant, a trusted provider of cutting-edge on-chain and market data. Accurate data was available from March 30, 2019 to time of data collection (June 9, 2025).

Date	Bitcoin Close	Active Addresses	...	Open Interest	Puell Multiple	SOPR
6-9-25	110303.871227	893460.0	...	3.784251e+10	1.243001	1.013669
6-8-25	105782.295477	779831.0	...	3.604954e+10	1.337595	1.011710
6-7-25	105645.525274	787698.0	...	3.601397e+10	1.245541	1.013583
6-6-25	104406.924809	859500.0	...	3.543465e+10	1.152875	1.006889
6-5-25	101613.260806	866463.0	...	3.385101e+10	1.156797	1.002772

Table 2: Example on-chain data from most recent timeframe.

Visualization of the relationships between Bitcoin closing price and these on-chain metrics can be achieved using a correlation matrix, which reveals latent linear relationships between variables. It is a major step in the feature engineering process, through which the curse of dimensionality – too many factors leading to overfitting and unnecessary complexity – is mitigated.

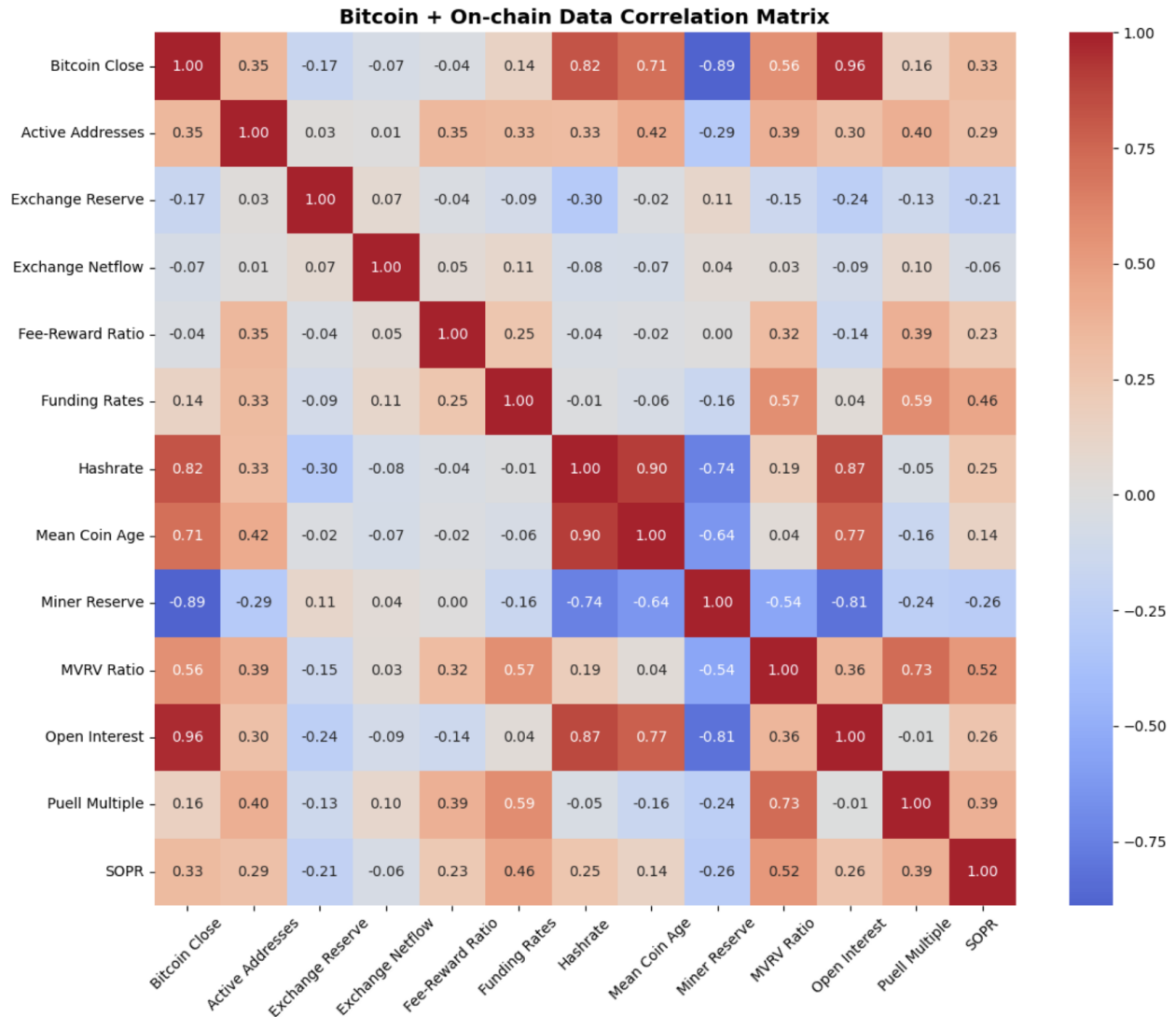


Figure 1: Correlation matrix for on-chain data.

Many interesting relationships between different factors can be observed here, but the top row is of main concern, which shows correlation specifically between Bitcoin close price and each on-chain factor. From this figure, Hashrate (cor 0.82), Mean Coin Age (cor 0.71), Miner Reserves (cor -0.89), and Open Interest (cor 0.96) are the variables with highest correlation magnitudes. This provides a comprehensive view of linear relationships between on-chain data, which informs feature selection.

On-Chain Factors Used in Predictive Modeling

The following on-chain metrics were leveraged as explanatory variables in machine learning models to forecast directional changes in Bitcoin's price, backed by higher correlations.

Miner Reserve

- **Definition:** Miner reserve denotes the total bitcoin held within wallets associated with mining entities.
- **Insight:** This value reflects the portion of newly-mined coins yet to be liquidated. High miner reserves may indicate an expectation for higher future prices, while a decline (miners selling coins) could foreshadow selling pressure and potential downward price movement.

Hashrate

- **Definition:** Hashrate is the average computational power employed by all miners, measured as hashes per second (commonly in exahashes).
- **Insight:** Higher hashrate enhances network security and is often interpreted as miner confidence in future profitability. Significant changes in hashrate can signal shifts in mining economics or sentiment.

Mean Coin Age (MCA)

- **Definition:** Mean Coin Age measures the average duration (weighted by value) that UTxOs remain unspent.
- **Computation:** MCA is the mean of products between each UTxO value and its days since creation.
- **Insight:** An increasing MCA suggests coins are being held rather than spent, indicating accumulation. Conversely, a sharp decline in MCA signals older coins are being moved, which is often correlated with heightened market activity or potential selloffs.

Open Interest

- **Definition:** Open Interest is the aggregate number of open positions (both long and short) in derivative markets.
- **Insight:** Rising open interest typically aligns with increased liquidity, volatility, and speculative capital entering the market. Sudden declines may precede long/short squeezes, amplifying price swings.

Brief Overview of Excluded Factors

The following on-chain or derivative metrics, while informative, were excluded from most final models due to redundancy, lower predictive power, or diminished relevance:

- *Exchange Reserve:* Tracks the total amount of bitcoin held on exchanges. Higher reserves often signal increased selling risk, but can be confounded by inter-exchange transfers.
- *Puell Multiple:* Ratio of daily bitcoin issuance value (USD terms) to its 365-day moving

average. Historically used to identify miner-driven market cycles.

- *Fee-Reward Ratio*: The fee component of block rewards as a percentage of total block rewards, reflecting transaction demand and network congestion.
- *SOPR (Spent Output Profit Ratio)*: Measures realized profit by comparing output values at spend time to those at creation. $SOPR > 1$ indicates coins are moved at a profit, useful for assessing market sentiment.
- *Active Addresses*: The unique count of active sending and receiving addresses within a timeframe, often considered as a proxy for network usage and adoption.
- *Exchange Netflow*: The net difference between bitcoin entering and leaving exchange addresses. Large net inflows are frequently associated with selling interest.
- *Funding Rates*: Periodic payments exchanged between long and short traders to align perpetual futures prices with spot prices. These rates can reflect trading bias or momentum extremes.
- *MVRV Ratio (Market Value to Realized Value)*: Compares total market capitalization to realized capitalization, helping to contextualize periods of over- or under-valuation in the market. ****Was used for support vector machines**

Market Data

While on-chain data has been thoroughly analyzed in relation to Bitcoin pricing, also of great interest are the co-movements between cryptocurrencies and major markets / indices. Will Bitcoin prices be influenced by fluctuations in the S&P 500? Why have so many people been juxtaposing Bitcoin and gold? This study aims to answer these questions, and provide further context on latent relationships between Bitcoin and other indices, commodities, and forex markets. As found in Table 1, a variety of these factors have been selected, and data from Bloomberg has been compiled from January 02, 2017 to time of data collection (June 9, 2025).

Date	Bitcoin Close	S&P Close	...	Gold Close	Nat Gas Close	EUR/USD
6-10-25	108923.014410	6038.81	...	3323.69	3.533	1.1425
6-9-25	110303.871227	6005.88	...	3326.19	3.635	1.1422
6-6-25	104406.924809	6000.36	...	3310.42	3.784	1.1397
6-5-25	101613.260806	5939.30	...	3352.65	3.677	1.1445
6-4-25	104724.591994	5970.81	...	3372.72	3.716	1.1417

Table 3: Example market data from most recent timeframe.

Again, as previously done with on-chain metrics, linear relationships in the data can better be

visualized through a correlation matrix. High correlations are expected between major indices and commodities, but what is less widely understood is how these major players relate to the Bitcoin market.

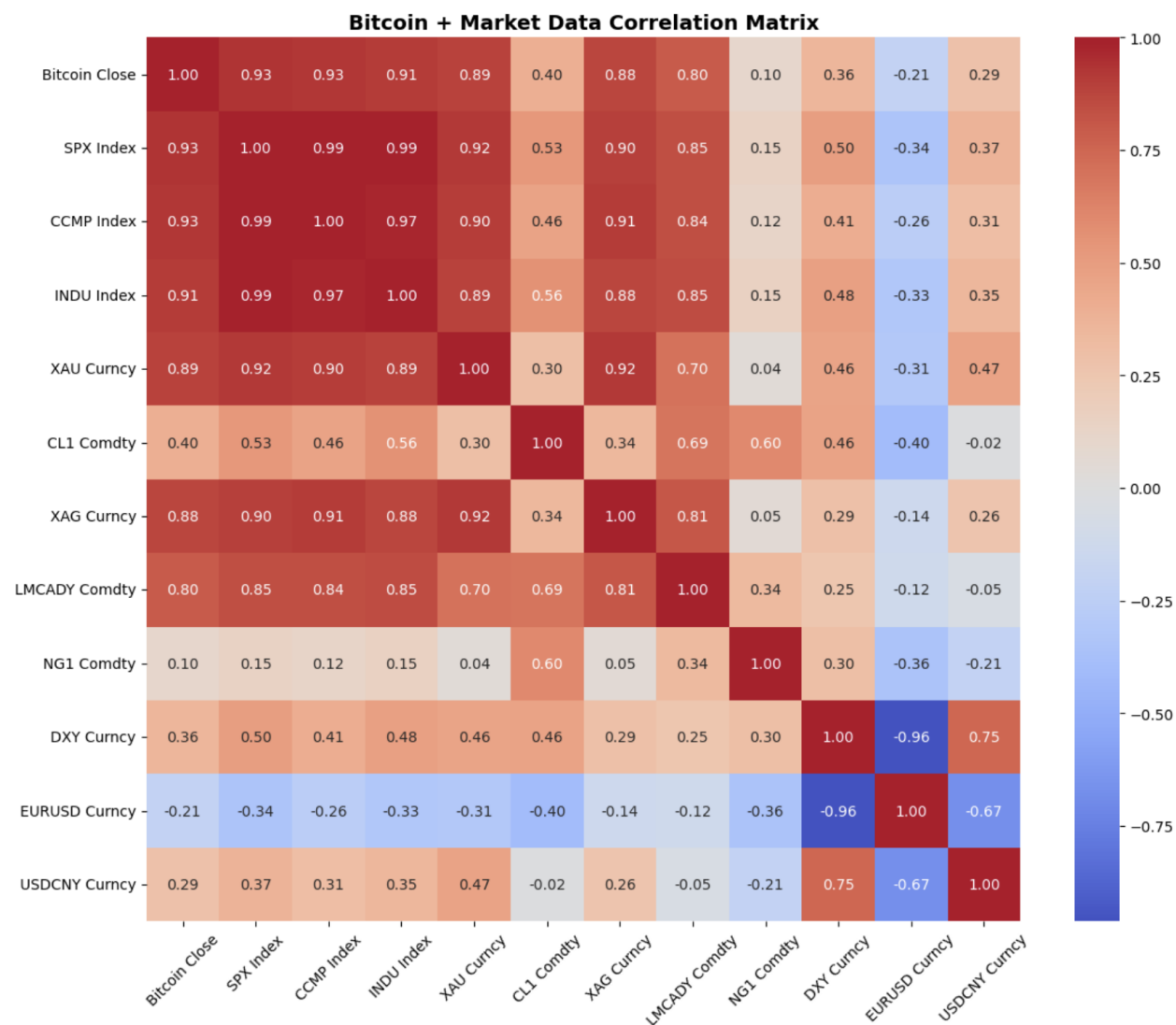


Figure 2: Correlation matrix for market data.

From the figure, it's clear that strong co-movement exists between Bitcoin and the three major indices: S&P 500, Nasdaq, and Dow Jones. Additionally, high correlation is found between Bitcoin and commodities such as gold, silver, and copper. However, few notable results are found in relation to forex markets. While high correlations are promising, it remains to be seen if any of these markets can serve as leading indicators for Bitcoin fluctuations in an effective model.

Macroeconomic Indicators

Finally, it may prove influential to include certain macroeconomic indicators in pricing models, as factors such as inflation, interest rates, or even geopolitical tensions could change how investors approach Bitcoin and cryptocurrency markets as a whole. Many macroeconomic indicators are collected quarterly, so after accessing data from Bloomberg, linear interpolation was performed to standardize data on a daily basis.

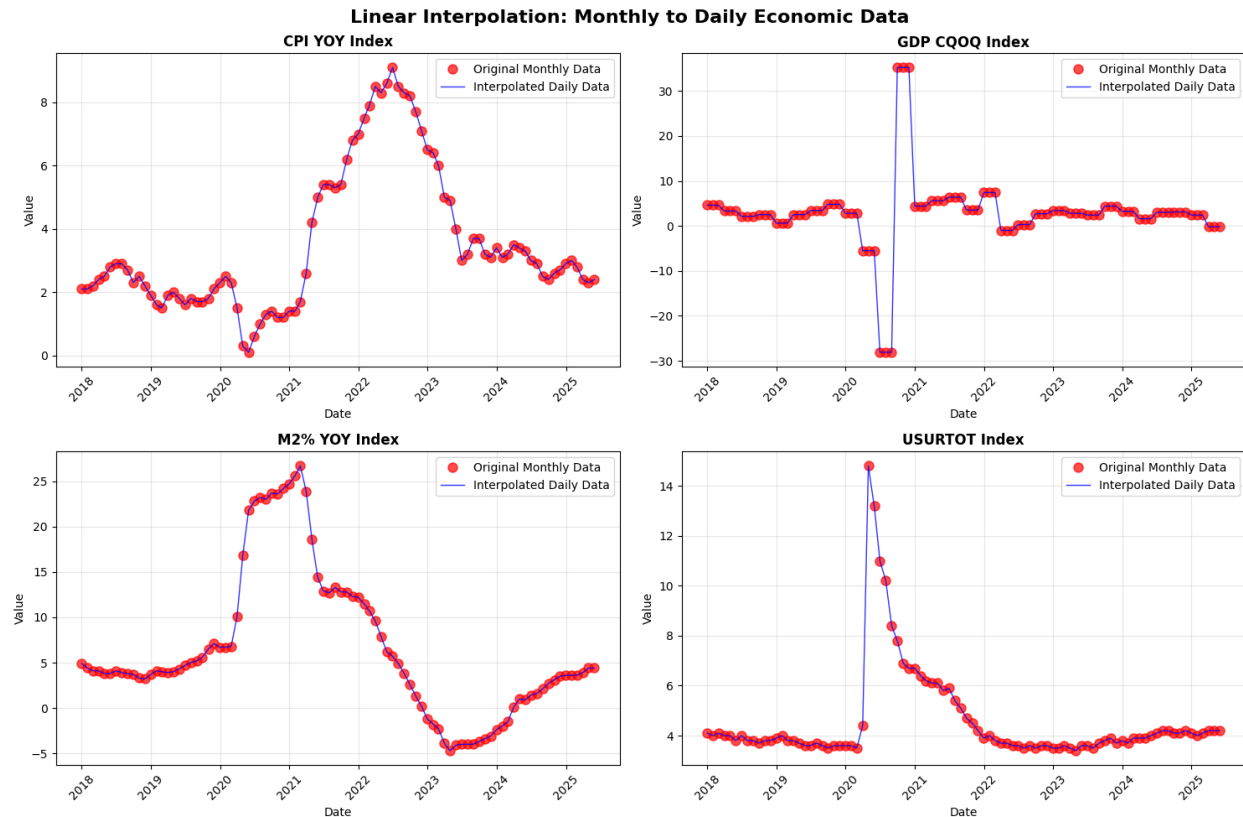


Figure 3: Linear interpolation of CPI YOY, GDP CQOQ, M2% YOY, and Unemployment Rate.

These four features, along with Federal Funds Target Rate (FDTR Index) and US 10 Year Treasury Yield (USGG10yr Index), comprised the macroeconomic indicators evaluated in the study. Linear interpolation was not required for the additional two features. The following correlation matrix is computed.

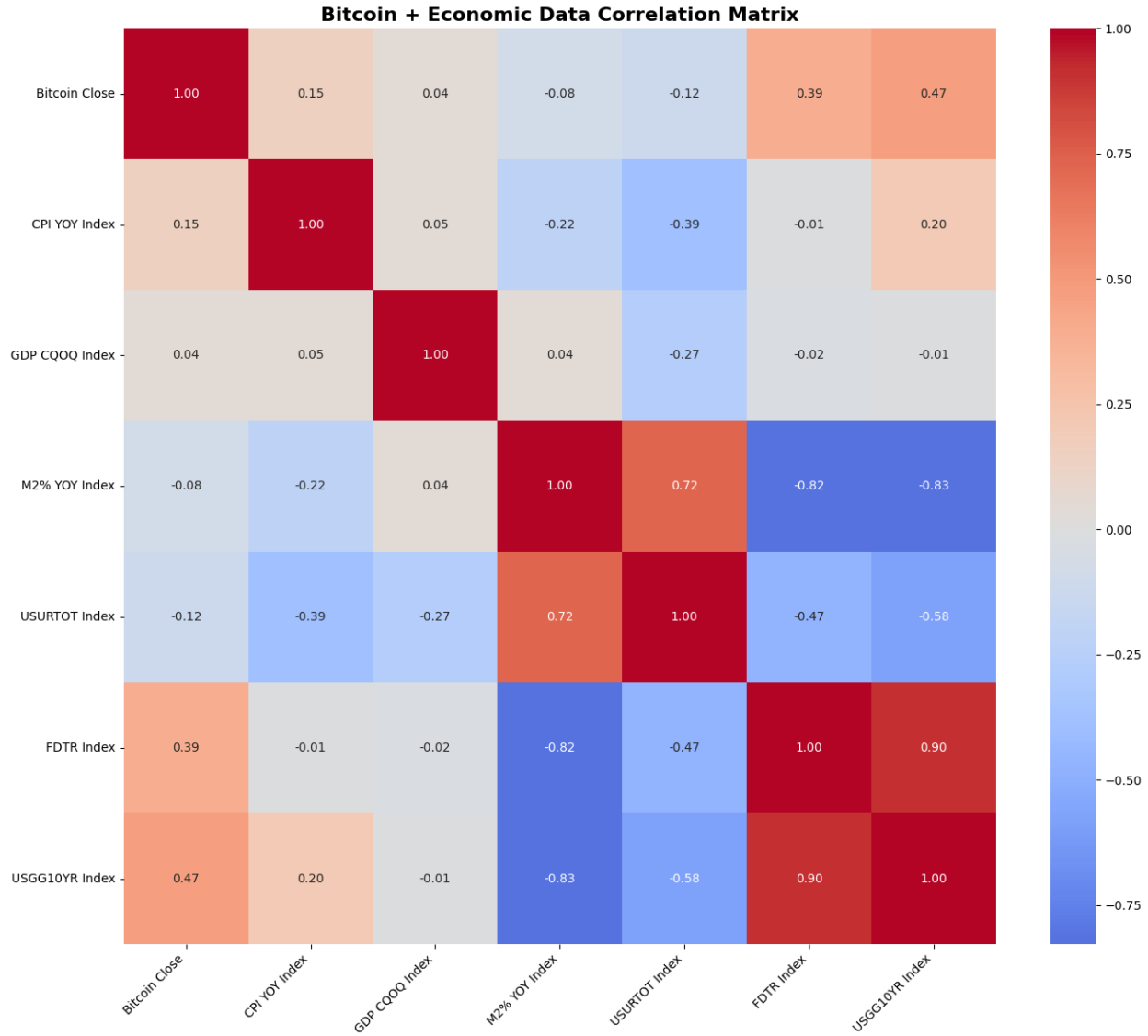


Figure 4: Correlation matrix for macroeconomic indicators.

Evidently, there is much less to take away here. But due to the fragile nature of the data, it is more difficult to find robust linear relationships between these macroeconomic indicators and Bitcoin price (it would more likely be a quick shock anyway).

Feature Engineering

All data has been collected for the purposes of this study, but we acknowledge that to achieve optimal results, an even greater magnitude of data would need to be compiled for expanded feature selection. For now, we've sampled the most significant factors from three categories, with potential for major predictive power.

However, the data must be effectively transformed before it can be input into different machine learning models, otherwise the results will be inconclusive.

Target Variable

The primary goal of the predictive models is to determine highest possible accuracy in forecasting **price direction** of Bitcoin close. This classification will be done as follows: for day d (the day from which our factors come), the target variable is 1 if the close price on day $d + 1$ is higher than on day d . Otherwise, if the close price on day $d + 1$ is less than day d , the target variable is 0. This can be represented mathematically as:

$$T_d = \max(0, \text{sign}(P_{d+1} - P_d)) \quad (1)$$

This effectively reflects the study's use of day d 's values to forecast behavior on day $d + 1$, ensuring there is no target leakage.

Factor Transformations

To align with the goal of capturing Bitcoin price direction, data must be transformed to indicate the dynamics of each on-chain metric, market, index, or macroeconomic indicator. Therefore, a multitude of transformations were applied to various factors, prior to input into machine learning models. Here is a succinct description:

- **On-Chain Data:** one-day percent change, three-day percent change, seven-day moving average, binary flags
- **Market Data:** daily percent returns (one-day percent change)
- **Economic Indicators:** one-day delta, raw form

Numerous combinations of these transformations were applied, in order to achieve highest testing set accuracy. The main intuition is simply that the study aims to capture dynamic relationships; this is best accomplished through the above transformations, which were individually evaluated and selected for each factor.

Boruta Feature Selection

Correlation matrices capture linear relationships between variables. However, the study also aims to evaluate non-linear relationships that may provide context towards Bitcoin’s price movements: the attempt to select most impactful non-linear factors is done through Boruta feature selection.

Essentially, Boruta is a wrapper algorithm around a Random Forest, a machine learning classification method that will be explored later in this paper. Boruta runs many trials to test each individual feature, but additionally adds *shadow features*, which are randomized versions of each feature that essentially compete against their original versions. Only features *more influential* than their random vectors are awarded points during each trial; finally, the Boruta model will score each feature on a binomial distribution to determine significance.

Separate Boruta algorithms, each with a total of 50 trials, were run for on-chain factors and market factors.

Boruta Factors	Random Baseline	Random Forest Cross-Validation Score
On-Chain Data	0.512	0.504 ± 0.028
Market Data	0.512	0.492 ± 0.023

Table 4: Accuracy of underlying random forests with transformed features.

The random forest accuracies are not too promising – this indicates the relationships between the study’s factors and Bitcoin close may not be non-linear, in the vein captured by decision trees. However, the Boruta simulations are still run to check for feature importance.

On-Chain Boruta

Feature	Scoring
Mean Coin Age	28/50 (56.0%)
MVRV Ratio	18/50 (36.0%)
Open Interest	10/50 (20.0%)
Fee-Reward Ratio	7/50 (14.0%)
Hashrate	5/50 (10.0%)
Miner Reserve	1/50 (2.0%)
Exchange Reserve	0/50 (0.0%)
Puell Multiple	0/50 (0.0%)
SOPR	0/50 (0.0%)
Active Addresses	0/50 (0.0%)
Exchange Netflow	0/50 (0.0%)
Funding Rates	0/50 (0.0%)

Market Boruta

Feature	Scoring
Copper	14/50 (28.0%)
Gold	2/50 (4.0%)
Natural Gas	1/50 (2.0%)
S&P 500	0/50 (0.0%)
Nasdaq	0/50 (0.0%)
Dow Jones	0/50 (0.0%)
Oil	0/50 (0.0%)
Silver	0/50 (0.0%)
Dollar Index	0/50 (0.0%)
EUR-USD	0/50 (0.0%)
USD-CNY	0/50 (0.0%)

Table 5: Boruta feature selection frequencies for transformed on-chain and market data.

Observing the results above, very few features would be labeled as "significant" by the Boruta binomial distribution. But, considering the lower accuracy of the random forests in the previous table, the Boruta results must be regarded with a grain of salt. Another weakness in the study is the relatively small pool of factors from which Boruta can choose from: as an extremely precise algorithm, likely hundreds of on-chain / market factors would be required for Boruta to select a handful passing the significance test.

Overall, the study combines interpretations of correlation matrices and Boruta significance tests to select features, which can be found in Table 1. Now, let's use them to build predictive machine learning models.

Machine Learning Models

Now that comprehensive data transformations and feature engineering methods have been performed, it is time to select models and evaluate their results. Since this is a classification problem, the study will use four prominent machine learning methods:

- **Logistic Regression (LR)**: simple linear model that estimates probabilities using the sigmoid function for classification
- **Support Vector Machine (SVM)**: an algorithm that calculates the optimal hyperplane in n-dimensional space, which separates data into two categories of maximal width
- **Tree-Based Algorithms**: using a tree-based setup, these models divide feature space into hierarchical structures to make predictions, capturing abstract nonlinear patterns
- **Long Short Term Memory (LSTM)**: a type of recurrent neural network (RNN) that can recognize nonlinear long-term dependencies in time-series data

For all models, the train-test split was 80%-20%. This means that the training period was from April 10, 2019 to March 7, 2024. The testing period, on which the accuracy score is measured, was from March 8, 2024 to May 29, 2025. This comes out to about 1282 training days, and 320 testing days. Before all models were run, a standard scaler was applied to the various data.

1 Logistic Regression

Methodology

Since the goal is to predict a binary variable, logistic regression is the perfect supervised machine learning algorithm to act as a baseline for the study. Essentially, logistic regression is built over a linear model, applying the sigmoid function to output a value between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

The sigmoid function is a classic activation function in machine learning. After applying the sigmoid, the threshold value to separate categories is usually 0.5.

$$\hat{T}_d = \begin{cases} 1 & \text{if } \sigma(z) \geq 0.5 \\ 0 & \text{if } \sigma(z) < 0.5 \end{cases}$$

But how is z calculated? The value is initialized as a linear combination of inputs, which were previously selected using feature engineering.

$$z = w^T \cdot X + b \quad (3)$$

These features are standardized and represented in matrix X . Meanwhile, vectors w and b serve as a set of weights and biases, which are used to transform the input matrix into output z . However, the weights and biases need to be optimized, otherwise the model will perform terribly. To evaluate the current performance of weights w and biases b , a function called **binary cross entropy**, or **log loss**, is employed.

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)] \quad (4)$$

Here, N is the number of observations, p_i is the probability outputted by the sigmoid function for day i , and y_i is the actual binary classification target variable on day i . The goal of logistic regression is to minimize average binary cross entropy, by recomputing w and b accordingly. To achieve this, let's apply **gradient descent**:

$$w_{k+1} = w_k - \eta \cdot \frac{\partial L}{\partial w} \quad (5)$$

$$b_{k+1} = b_k - \eta \cdot \frac{\partial L}{\partial b} \quad (6)$$

To calculate the weights and biases for iteration $k + 1$, we use the above formulas, relying on their previous values. The learning rate, η , is usually set to 0.01. The derivatives dL/dw and dL/db are calculated by applying the chain rule to binary cross entropy in terms of w and b .

$$L_{BCE}(w, b) = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\sigma(w^T x_i + b)) + (1 - y_i) \log(1 - \sigma(w^T x_i + b))] \quad (7)$$

Saving too much computation, the derivatives work out quite nicely.

$$\frac{\partial L}{\partial w} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \cdot x_i \quad (8)$$

$$\frac{\partial L}{\partial b} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i) \quad (9)$$

During logistic regression, weights w and biases b are iteratively updated using gradient descent until some optimization threshold. Python's scikit-learn library, used for this study, stops iteration when the gradient reaches below a convergence condition.

Results

After running the logistic regression and toggling different factors with various transformations, the highest accuracy score achieved was 55.94%, a considerable improvement from the random baseline. This indicates that significant predictive linear relationships exist between Bitcoin and the various market indices and on-chain metrics. The equation that reaches this accuracy score can be found below:

$$\hat{T}_d = \frac{1}{1 + e^{-z}} \quad (10)$$

where

$$\begin{aligned} z = & 0.066599 - 0.038974 \cdot M + 0.033381 \cdot H - 0.197812 \cdot A - 0.069620 \cdot I \\ & - 0.695995 \cdot P + 0.263836 \cdot Q + 0.280560 \cdot D + 0.097415 \cdot G - 0.044349 \cdot S \\ & - 0.082140 \cdot C - 0.078725 \cdot F_1 - 0.031576 \cdot F_2 + 0.076037 \cdot T_1 - 0.067600 \cdot T_2 \end{aligned} \quad (11)$$

As the equation displays, gradient descent applied to minimize binary cross entropy produces very precise weights. The variables, which correspond to matrix X , represent the following:

- M : Miner reserve 7-day delta
- H : Hashrate 7-day rolling mean
- A : Mean Coin Age 7-day pct change
- I : Open Interest daily pct change
- P : S&P 500 daily pct change
- Q : Nasdaq daily pct change
- D : Dow Jones daily pct change
- G : Gold daily pct change
- S : Silver daily pct change
- C : Copper daily pct change
- F_1 : Raw Fed Funds rate
- F_2 : Fed Funds one-day delta
- T_1 : 10Y treasury yield daily pct change
- T_2 : 10Y treasury yield 5-day delta

Additionally, the logistic regression F1 score was 0.6536, indicating meaningful ability to identify directional trends beyond a random baseline. Of course, as time progresses or if the dataset is expanded, this equation will need to be re-trained and the weights and biases modified accordingly. However, it shows that logistic regression can be quite an effective method, exhibiting latent linear relationships in the data.

2 Support Vector Machine

Methodology

Moving on from logistic regression, another great supervised machine learning method for binary classification is the support vector machine. This algorithm attempts to find an optimal hyperplane in n -dimensional space (where n is the number of factors) that separates data points into two categories for classification. It does so by calculating the maximum margin between the hyperplane and closest data points on either side, called **support vectors**. While humans obviously cannot visualize n -dimensional space, support vector machines can use a **kernel function** to map the hyperplane to higher-dimensional space. This kernel may be linear or non-linear, as shown below.

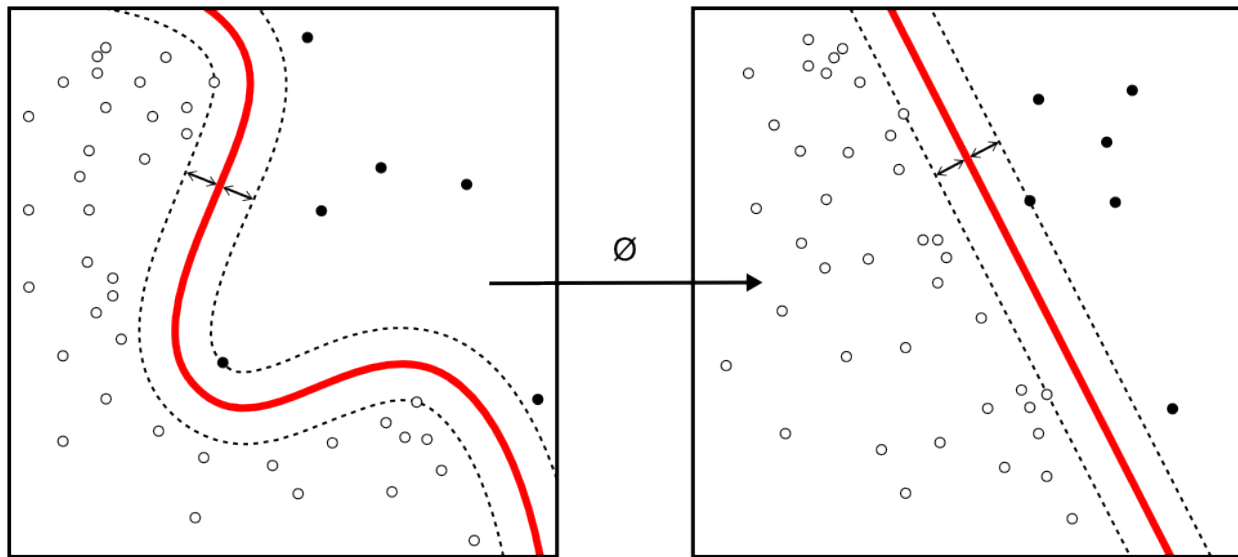


Figure 5: Examples of linear and non-linear kernels, visualized in the 2-dimensional plane.

All support vector machines, regardless of kernel function, aim to minimize the **regularized loss**

function

$$\min_f C \sum_{i=1}^n \mathcal{L}(f(x_i), y_i) + \Omega(w) \quad (12)$$

which essentially finds the boundary $f(X)$ that classifies data as correctly as possible, while maximizing the boundary between support vectors. The hinge loss $\mathcal{L}$ penalizes points that are incorrectly categorized compared to actual binary value y_i , while $\Omega(w)$ keeps the decision boundary as wide as possible. The formulas for $\mathcal{L}$ and $\Omega(w)$ are less significant for the purposes of this explanation. More importantly for this study, parameter C determines the balance between these goals: larger C prioritizes fewer classification errors above possible overfitting, while smaller C prioritizes a simpler, wider boundary.

This section will test two types of support vector machines: one with **linear kernels**, and one with an **RBF (radial basis function) kernel**. While a linear support vector machine simply attempts to maximize the distance between support vectors, the RBF kernel measures similarity between data points, allowing for curved boundaries. The linear hyperplane, much like linear and logistic regression, is given by

$$f(X) = w^T X + b \quad (13)$$

and plugging into the regularized loss function, the optimization problem is reached.

$$\min_{w,b} C \sum_{i=1}^n \max(0, 1 - y_i(w^T x_i + b)) + \frac{1}{2} \|w\|^2 \quad (14)$$

As seen in the equation, the value of C will determine which chunk of the equation is more urgent to minimize. Meanwhile, the RBF kernel maps its hyperplane to higher dimensional space with the following kernel function

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2) \quad (15)$$

where higher γ creates a more complex decision boundary. This parameter is predetermined through scikit-learn. The other important element in RBF kernels are dual coefficients, α_i 's, optimized to measure how much influence each support vector has on the curvature of the hyperplane. This is all combined into a dual optimization problem

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j K(x_i, x_j) \quad (16)$$

$$\text{subject to } \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \quad (17)$$

which essentially allows the hyperplane to capture more complex, non-linear patterns in the data. Both kernel functions will be assessed with varying parameter C , in attempts to capture the most effective SVM model.

Results

Support vector machines were run for both linear and RBF kernels, with parameter C being set to 1, 5, and 10. The respective accuracy scores are displayed in the table below.

	$C = 1$	$C = 5$	$C = 10$
Linear Kernel	0.5469	0.5375	0.5312
RBF Kernel	0.5062	0.5094	0.4938

Table 6: Accuracy scores for support vector machines.

Interestingly enough, these results show the most effective SVM achieves 54.69% accuracy, with a linear kernel that prioritizes establishing the simplest hyperplane (since $C = 1$). Again, this suggests that the relationships between the various features and the target variable are quite linear. The associated F1 score is 0.6588, showing a promising balance between precision and recall.

3 Tree-Based Algorithms

Methodology

The **decision tree** is the core tree-based model, which uses a series of nodes that represent partitions in which each data point could fall. Through a series of yes/no questions (usually inequalities for quantitative data), an optimal flow is created to maximize classification accuracy.

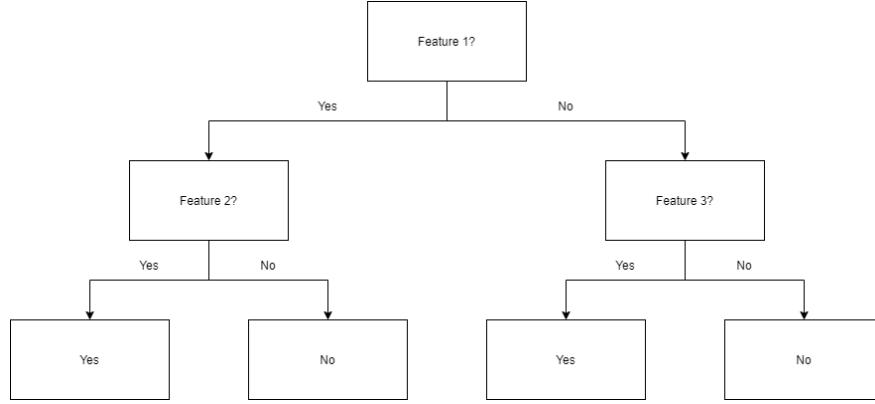


Figure 6: Visualization of a simplified decision tree.

The **depth** of a decision tree is defined as the length of the longest path from the root node to any leaf node. Larger depth increases complexity, but too much depth leads to overfitting. The decision tree model chooses nodes (to construct itself) based on a greedy algorithm, using **Gini impurity**. This metric essentially measures the probability that a randomly chosen data point would be incorrectly categorized at some factor's node. The factor with lowest Gini impurity is selected.

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2 \quad (18)$$

For each possible outcome i for a certain factor, the Gini index is found by subtracting the sum of the squared probabilities of all outcomes from 1. A lower Gini impurity score represents a more homogeneous distribution, and thus easier to categorize. The entire decision tree is constructed in this manner.

The **random forest** is a form of ensemble machine learning, because it combines a series of decision trees to make stronger predictions. For each individual tree in the forest, random features are selected, and data is resampled through bootstrapping. This ensures that no two decision trees are the same. The results from all decision trees are aggregated, and based on majority voting, the final classification result is obtained.

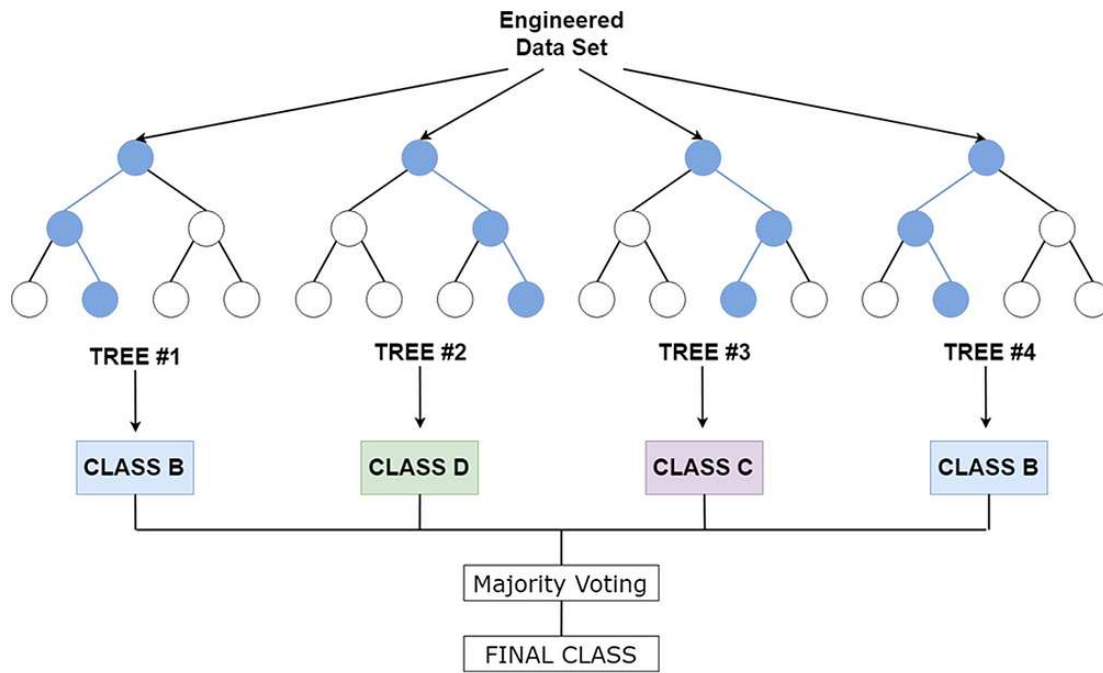


Figure 7: Random forests combine many decision trees with varied features and bootstrapped data.

Random forests are designed to reduce overfitting; however, with the smaller dataset used in this study, it may also overcomplicate the prediction process.

Results

First, multiple decision trees were trained and tested using varying maximum depth thresholds. Again, larger depth means increased complexity, which highlights the tradeoff between accuracy and overfitting.

Depth	max = 1	max = 3	max = 5	max = 10	max = None
Accuracy Score	0.5125	0.5312	0.4719	0.5375	0.4813

Table 7: Accuracy scores for decision trees.

The highest accuracy score of 53.75% is achieved at maximum depth of 10, which likely establishes the strongest balance between accuracy on training data and overfitting due to complexity. However, the associated F1 score is 0.5404, highlighting less effectiveness in classification compared to previous models.

Now, let's evaluate the results from various random forests, the series of decision trees in parallel. Again, this ensemble learning method is intended to reduce overfitting. The rows in the table below are divided by maximum depth of each decision tree, while columns are determined by the number of decision trees in the random forest.

	num trees = 50	num trees = 100	num trees = 200
max = 5	0.4969	0.4906	0.4969
max = 10	0.4938	0.4781	0.4813

Table 8: Accuracy scores for random forests.

Results here perform worse than the random baseline across the board – random forests are the most inaccurate model evaluated so far. This primarily signifies that this non-linear algorithm cannot effectively capture relationships in the data. There is also likely heavy overfitting occurring, which further throws the model's accuracy score downwards. Overall, tree-based algorithms seem to be a dead-end in terms of directional price prediction.

4 Long Short Term Memory (LSTM)

Introduction

This section of this study is dedicated to deep learning in Bitcoin price prediction, which has been a great topic of interest recently in financial machine learning. We'd like to begin by acknowledging that effective deep learning models require astronomical amounts of data for proper prediction. This study includes around 18,000 data points across 1,280 days, which definitely does not provide enough training data to receive optimal results. However, building these networks is an interesting and valuable exercise nonetheless.

Theory

Long short term memory (LSTM) is an augmented form of **recurrent neural networks (RNNs)**, which are a growingly popular form of neural networks that are optimized for sequential data. This is because recurrent neural networks contain feedback loops, which allow the network to access a *history* of sorts when making future predictions.

Take large language models (LLMs), for example, which include OpenAI's ChatGPT or Google's

Gemini. These LLMs are built upon **natural language processing (NLP)**, the broad subsection of artificial intelligence that allows computers to interpret and generate human languages. An essential problem in NLP is word generation; in context, this might mean an LLM is given an in-progress sentence, and has to generate the next word. For example,

At RCM, I wrote a research paper on -----.

If we used a regular neural network to generate the next word, we'd have to input the entire sentence to get a reasonable result. However, this would require re-vectorizing the sentence each time a new word is generated, which is very inefficient. And if we don't provide the input sentence, the LLM will generate a random word, for example

At RCM, I wrote a research paper on toasters.

Of course, this is absurd. However, recurrent neural network architecture allows for remembrance of previous states – for example, the word *RCM* is crucial, and the RNN will remember this context to generate a meaningful next word

At RCM, I wrote a research paper on Bitcoin.

Immediately, it is noticeable how the ability of RNNs to remember historical data can be extremely useful for stock prediction. LSTM architecture leverages RNNs to accomplish that very goal.

Recurrent Neural Networks

The differentiator from traditional feedforward neural networks, recurrent neural networks possess a **hidden state** at each neuron, in the form of a feedback loop that accumulates memory from all previous elements in a given sequence.

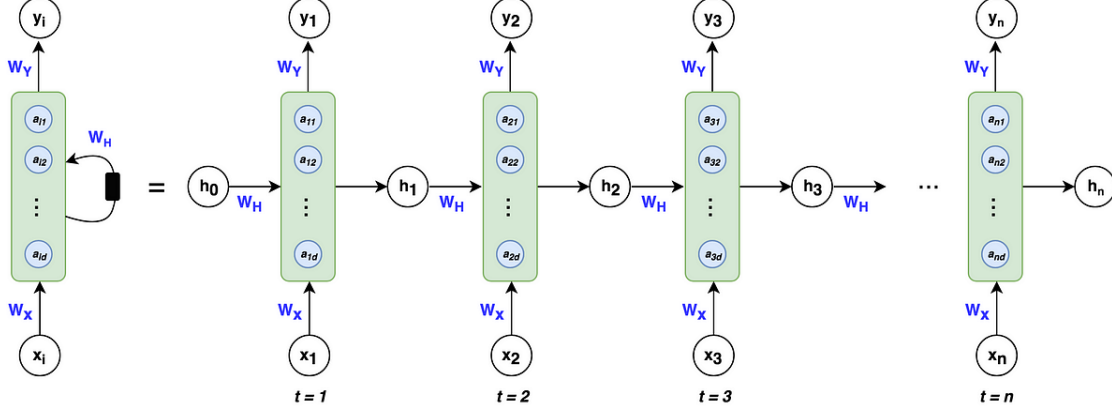


Figure 8: Recurrent neural network architecture.

This hidden state allows the neural network to track dependencies over time, making RNNs very useful in natural language processing and time-series prediction. Each hidden state h_t is a function of the input x_t and the previous hidden state, h_{t-1} . Therefore, we get a chain of influence, connecting the entire neural network. And, output y_t is a simple function of optimized weights and hidden state h_t .

However, while RNNs incorporate the element of memory using hidden states, a major issue is encountered when many layers are required. To optimize the neural network, a process called **backpropagation through time** is used to apply gradient descent and update the weight matrices that calculate hidden and output states. Using the chain rule for backpropagation,

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \cdot \prod_{k=t+1}^T \frac{\partial h_k}{\partial h_{k-1}} \quad (19)$$

Since the tanh function limits values to the interval $(-1, 1)$, usually $|\frac{\partial h_k}{\partial h_{k-1}}| < 1$, and after many multiplications, $\frac{\partial L}{\partial h_t}$ becomes negligible. All of this is to say, earlier inputs in the RNN have much less influence than desired. This is called the **vanishing gradient problem**, the main motivating factor behind LSTMs.

Long Short Term Memory

The fix for the vanishing gradient problem? Long short term memory recurrent neural networks. Rather than relying on a single hidden state per neuron, LSTMs also incorporate a memory cell for long-term remembrance.

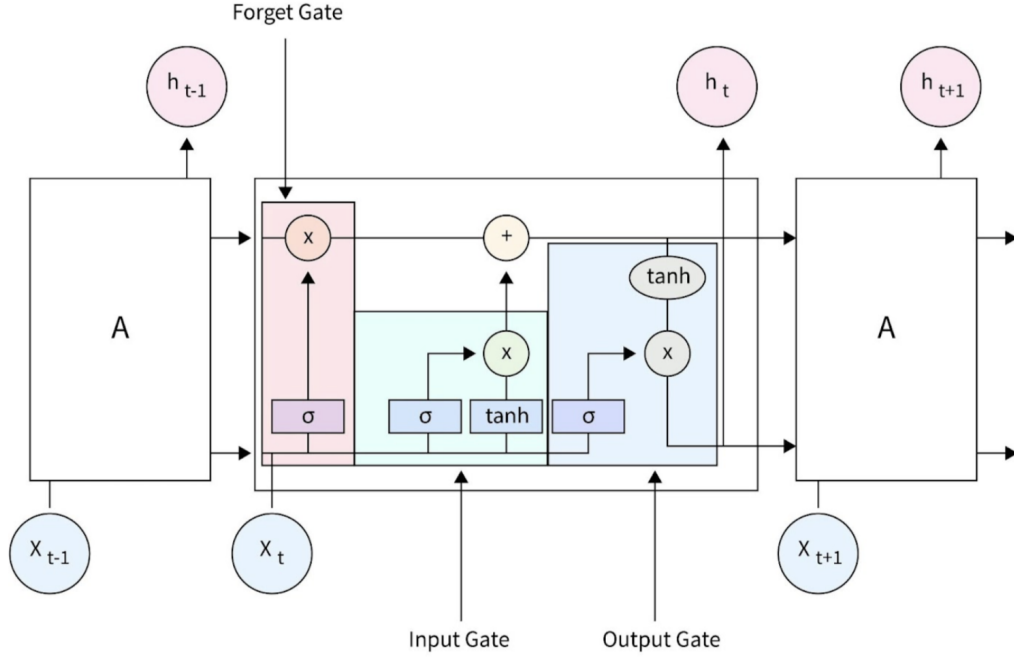


Figure 9: Long short term memory architecture.

Three main gates comprise the LSTM architecture. The first is the **forget gate**, which removes sequential information that the model deems no longer useful. Its inputs are h_{t-1} , which is the previous cell's hidden state, and x_t , the current cell's input. Using a sigmoid, the binary outcome determines if the information is important or not.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (20)$$

Note that throughout these equations, W refers to weights and b refers to biases that are iteratively updated through backpropagation. If $f_t = 0$, the information is forgotten in the next step; if $f_t = 1$, it is remembered.

Next in sequence is the **input gate**, which decides how information is added to the cell state. Essentially, a filter is created to manipulate values h_{t-1} and x_t . Then, the input gate is used to calculate an updated cell state given by

$$C_t = f_t \odot C_{t-1} + i_t \odot \hat{C}_t \quad (21)$$

where

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (22)$$

$$\hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (23)$$

and $\odot$ refers to element-wise multiplication for matrices of the same dimension. Finally, the **output gate** controls which part of the cell is most influential in output,

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (24)$$

and this is used to compute a final hidden state.

$$h_t = o_t \odot \tanh(C_t) \quad (25)$$

Basically, these series of gates simply act as filters for important versus unimportant memory; retaining and updating previous results to capture the effect of long term memory (as opposed to RNNs, which cannot). In current research, LSTM models are being coupled with many different feature engineering methods to time-series tasks like stock price prediction.

Results

Inherently, the training of LSTMs has an element of stochasticity, because weights are randomly initialized, TensorFlow randomly shuffles data between epochs, and in this study, random neurons are selected for deactivation during dropout to prevent overfitting. Therefore, ten trials of LSTM training and prediction are executed, with the means and standard deviations found below.

Training Accuracy	Testing Accuracy
0.5749 ± 0.0209	0.5114 ± 0.0128

Table 9: Accuracy of long short term memory across ten trials.

As expected, the LSTM doesn't perform as strongly as some of the previous models, likely due to the simple fact that the dataset is not large enough for optimal performance of deep learning. Yet, high potential can already be identified, as the LSTM yields the greatest training accuracy of any model in the study. Testing accuracy would increase with more observations, as this reduces the chance of overfitting and prevents the early stopping mechanism from halting the deep learning network too quickly.

Conclusion and Future Recommendations

Findings and Commentary

This original research paper represents the first ever machine learning study conducted on Bitcoin pricing that combines *both* on-chain metrics and market conditions for predictive analysis.

Often, professional traders aim for directional accuracy of 55%-60% in their trades, and considering the volatility of cryptocurrencies, this is an ambitious range. The logistic regression model achieved greatest accuracy across the testing period at 55.94%, quite an impressive marker for a linear-based model. Further research would be required to engineer a profitable trading strategy from these baseline predictions. While more complicated models like long short term memory produced decreased accuracy, this can likely be attributed to the smaller size of the dataset. However, running all these machine learning models in parallel acts as a strong blueprint to understand the preliminary nature of a volatile asset, and its degree of linearity (or non-linearity).

This analysis seems to show that Bitcoin is primarily acting like a stock index, such as the SPX, Nasdaq or Dow; and secondarily, a metal commodity. The classification of Bitcoin as most like a stock index makes this a more transparent product for mainstream investors and portfolio managers, who previously avoided the asset class due to ambiguity. While a mainstream investor may still want to avoid spot bitcoin to avoid uncertainty and cumbersomeness on physical asset custody; there are traditional financial instruments, such as Bitcoin Futures (launched in 2017) and Bitcoin Future ETFs (launched in 2021) as well as options on these products that eliminate the issue of direct spot investment. A portfolio manager can allocate to Bitcoin Futures and/or Bitcoin ETFs much like investing in the ES futures and/or SPY ETF. While more on-chain data would need to be pulled and implemented into a logistic regression and/or long short term memory recurrent neural networks in future, our findings indicate diminished importance compared to Bitcoin's relationship with established markets. We would specifically like to highlight Bitcoin's notable relationship with copper, a definite point of further investigation.

Future Research

Most obviously, expanding the dataset from this study (specifically in regards to on-chain data) while implementing the same methodologies could produce further promising results. With a

larger dataset and more factors, both Boruta feature selection and long short term memory (LSTM) will be augmented, increasing predictive power. Then, most significantly, **trading strategies** can be developed based on the signals from this machine learning study, which can ideally provide an edge as Bitcoin markets become further optimized. Backtesting can be done on Bitcoin historical data, with the end goal being implementation of various ML-influenced trading strategies.

Another interesting avenue to explore would be intraday Bitcoin predictive pricing, which would involve looking at fluctuations in extremely small periods of time. This could lead to high frequency trading in cryptocurrency markets, which is still a novel and extremely risky undertaking. However, an increasing number of firms are investigating the integration of machine learning into short-term prediction and trading strategies.

Finally, another area of influence in Bitcoin pricing could be electricity demand and generation, which relates to ASIC usage, pricing, and other elements of Bitcoin's sophisticated mining processes. Through analysis of the infrastructure, better understanding of pivotal factors to enhance machine learning models could be discovered.

References

- [1] Oluwadamilare Omole, David Enke. *Deep Learning for Bitcoin Price Direction Prediction: Models and Trading Strategies Empirically Compared*. 2024.
<https://doi.org/10.1186/s40854-024-00643-1>
- [2] Azaz Hassan Khan et al. *A performance comparison of machine learning models for stock market prediction with novel investment strategy*. 2023.
<https://doi.org/10.1371/journal.pone.0286362>
- [3] Franco Valencia et al. *Price Movement Prediction of Cryptocurrencies Using Sentiment Analysis and Machine Learning*. 2019. <https://doi.org/10.3390/e21060589>
- [4] scikit-learn. *User Guide*. 2025.
https://scikit-learn.org/stable/user_guide.html
- [5] GeeksforGeeks. *Machine Learning*. 2025. <https://www.geeksforgeeks.org/machine-learning/machine-learning/>